SLIIT UNI

THE KNOWLEDGE UNIVERSITY

BSc (Hons) in Information Technology – SE

Year 3 – Semester 2

AI-Enabled Smart Healthcare Appointment & Telemedicine Platform

Submitted By: Y3S2_WE_14

	Registration Number	Name
1	IT23150416	Jayasingha J.A.V
2	IT23222786	Budara V.P. R
3	IT23285156	Bandara A.M.H.V.R
4	IT23396272	Kariyawasam L.L.N

Malabe Campus

Git Hub Repository: *DS_SLIIT_Project*

Table Of Contents

1.Abstract

This report presents the design and implementation of a cloud-native, AI-enabled Smart Healthcare Appointment and Telemedicine Platform developed as part of the Distributed Systems module. The platform is inspired by real-world telemedicine systems such as Channeling.lk and oDoc and is designed to enable patients to book doctor appointments, attend video consultations, upload medical documents, and receive AI-based preliminary health suggestions.

The system is built entirely on a Microservices architecture, comprising eight independent services: an Authentication Service, Patient Management Service, Doctor Management Service, Appointment Service, Telemedicine Service, Payment Service, Notification Service, and an AI Symptom Checker Service. All services are containerized using Docker and orchestrated using Kubernetes. An API Gateway acts as the single-entry point, routing requests to the appropriate downstream service. Asynchronous inter-service communication is achieved through Apache Kafka, enabling event-driven workflows such as notifications on appointment booking, approval, and payment confirmation.

The platform integrates multiple third-party technologies: Jitsi Meet for real-time video consultations, Stripe (sandbox) for secure online payment processing, SMSAPI.LK for SMS notifications, Nodemailer (SMTP) for email notifications, and a custom Decision Tree machine learning model trained on a medical symptom dataset for AI-powered symptom analysis. Role-based access control is enforced across all services using JSON Web Token (JWT) authentication, supporting three user roles: Patient, Doctor, and Admin.

The frontend is developed using React.js (Vite) and communicates with backend services exclusively through the API Gateway. The system is fully deployable via Docker Compose for development and Kubernetes manifests for production-grade deployment, with Prometheus and Grafana integrated for real-time monitoring and observability.

2.Introduction

The rapid digitization of healthcare services has created a growing demand for accessible, scalable, and secure telemedicine platforms. Traditional appointment systems are hindered by manual processes, geographic barriers, and limited scalability. Platforms such as Channeling.lk, oDoc, and mHealth have demonstrated that digital healthcare solutions can significantly improve patient outcomes by connecting patients with qualified medical professionals remotely, at any time.

This project addresses these challenges by developing a full-stack, cloud-native Smart Healthcare Appointment and Telemedicine Platform. The platform is designed to serve three categories of users: patients who require convenient access to medical consultations; doctors who need tools to manage their availability, conduct virtual consultations, and issue digital prescriptions; and administrators who oversee platform operations, verify doctor credentials, and manage user accounts.

The core objective of this assignment is to apply distributed systems principles specifically the Microservices architectural pattern to design a system that is independently deployable, scalable, fault-tolerant, and maintainable. Each functional domain of the healthcare platform is implemented as a separate, self-contained service with its own database, following the principle of data isolation central to microservices design. These services communicate with each other both synchronously via RESTful HTTP and asynchronously via Apache Kafka, ensuring loose coupling and high availability.

The platform incorporates several layers of technology to fulfil the assignment requirements. RESTful APIs are implemented using Node.js and Express.js. The React.js frontend provides an asynchronous, single-page application interface. Docker containers package each service for consistent deployment, and Kubernetes manages container orchestration on a scale. Security is enforced through JWT-based authentication, role-based access control, and HTTPS-ready configurations.

An optional but fully implemented enhancement is the AI Symptom Checker Service, which exposes a trained Decision Tree classification model via a Python Flask microservice. The model, trained on a structured medical symptom dataset using scikit-learn, accepts patient-reported symptoms and returns probable medical conditions alongside a recommended doctor specialty, enabling patients to make more informed decisions when booking consultations.

The following sections of this report describe the high-level architecture of the system, the service interfaces exposed by each microservice, the key workflows and their design, the authentication

and security mechanisms adopted, and the individual contributions of each group member. A full code appendix is included at the end of the report.

3. System Overview

3.1 High Level Description of the system

MediConnect is a cloud-native, AI-enabled Smart Healthcare Appointment and Telemedicine Platform built using the Microservices architectural pattern. The platform provides an end-to-end digital healthcare experience by connecting patients with qualified, verified doctors through an interactive web application. It enables patients to discover doctors, book appointments, make secure online payments, attend real-time video consultations, upload and manage medical documents, receive digital prescriptions, and obtain AI-powered preliminary health assessments all within a single unified platform.

The system is composed of eight independent backend microservices, each responsible for a distinct functional domain. All client requests are routed through a centralized API Gateway, which acts as the single-entry point and proxies requests to the appropriate downstream service. Inter-service communication is handled through two mechanisms: synchronous RESTful HTTP calls via Axios for real-time data retrieval, and asynchronous event-driven messaging via Apache Kafka for non-blocking notifications and workflow triggers.

Each microservice maintains its own dedicated PostgreSQL database, enforcing strict data isolation in line with microservices best practices. All services are containerized using Docker and are fully deployable on Kubernetes. Real-time system observability is provided through Prometheus metrics collection and Grafana dashboards integrated into each service.

The front end is a React.js single-page application (built with Vite) that communicates exclusively with the backend through the API Gateway. The platform additionally integrates Google Gemini AI as a conversational health assistant, accessible to patients via a chat bubble on the dashboard.

3.2 Core functionalities

The platform is organized around the following core functional modules:

Patient Management

- Patient self-registration and profile management
- Upload of medical reports and documents (PDF, JPEG, PNG - up to 20 MB)
- View personal medical history and uploaded records

- View prescriptions issued by doctors

Doctor Management

- Doctor profile management and weekly availability schedule configuration
- Credential upload and submission for admin verification
- Accept or reject incoming appointment requests
- View patient-uploaded medical records during consultation
- Issue and manage digital prescriptions

Appointment Booking

- Browse approved doctors filtered by speciality
- View real-time slot availability for any given week
- Book, cancel, and track appointment status
- In-appointment messaging between patient and doctor
- Automated appointment reminders via the scheduler

Telemedicine (Video Consultation)

- Automatic creation of a Jitsi Meet video session upon appointment approval
- Patients and doctors join video calls directly from their dashboards
- Session lifecycle management (start, end, status tracking)

Payment Processing

- Stripe (sandbox) payment intent creation on appointment booking
- Client-side payment confirmation via Stripe.js
- Stripe webhook handler for asynchronous payment status updates
- Doctor revenue reports and admin financial statistics

Notification System

- Event-driven SMS notifications via SMSAPI.LK
- Email notifications via Nodemailer (SMTP/Gmail)
- In-app real-time notification bell with mark-as-read functionality
- Notifications triggered for: registration, appointment booking, approval, rejection, payment confirmation, and doctor verification

AI Symptom Checker

- Patients input symptoms through the SymptomChecker component
- A Node.js service forwards symptoms to a Python Flask microservice
- The Flask service runs a trained **Decision Tree classifier** (scikit-learn) on a structured medical symptom dataset
- Returns probable conditions, confidence scores, and a recommended doctor specialty

AI Health Chat Assistant

- A persistent chat bubble available on all dashboards
- Powered by **Google Gemini 2.5 Flash** API

- Acts as a general health information assistant - advises on symptoms, medications, and when to seek urgent care
- Scoped with a system prompt to remain medically responsible and remind users to consult professionals

Admin Operations

- User management: list, search, filter, activate/deactivate user accounts
- Doctor verification: review submitted credential documents, approve or reject registrations
- Platform-wide financial statistics and audit logs

3.3 Target Users

The platform serves three distinct user roles, each with a tailored dashboard and permission set:

Role	Description
Patient	Individuals seeking medical consultation. Can register, browse doctors, book and pay for appointments, attend video consultations, upload medical records, view prescriptions, use the AI symptom checker, and chat with the Gemini health assistant.
Doctor	Licensed medical professionals. Can manage their profile and weekly availability schedule, submit credentials for platform verification, accept or reject appointments, conduct Jitsi video consultations, view patient medical records, issue digital prescriptions, and track consultation revenue.
Admin	Platform administrators. Can manage all user accounts, verify doctor registrations by reviewing uploaded credentials, approve or reject doctor applications, view platform-wide financial statistics, and access audit logs of system activity.

4. System Architecture

4.1 Architecture Style - Microservice

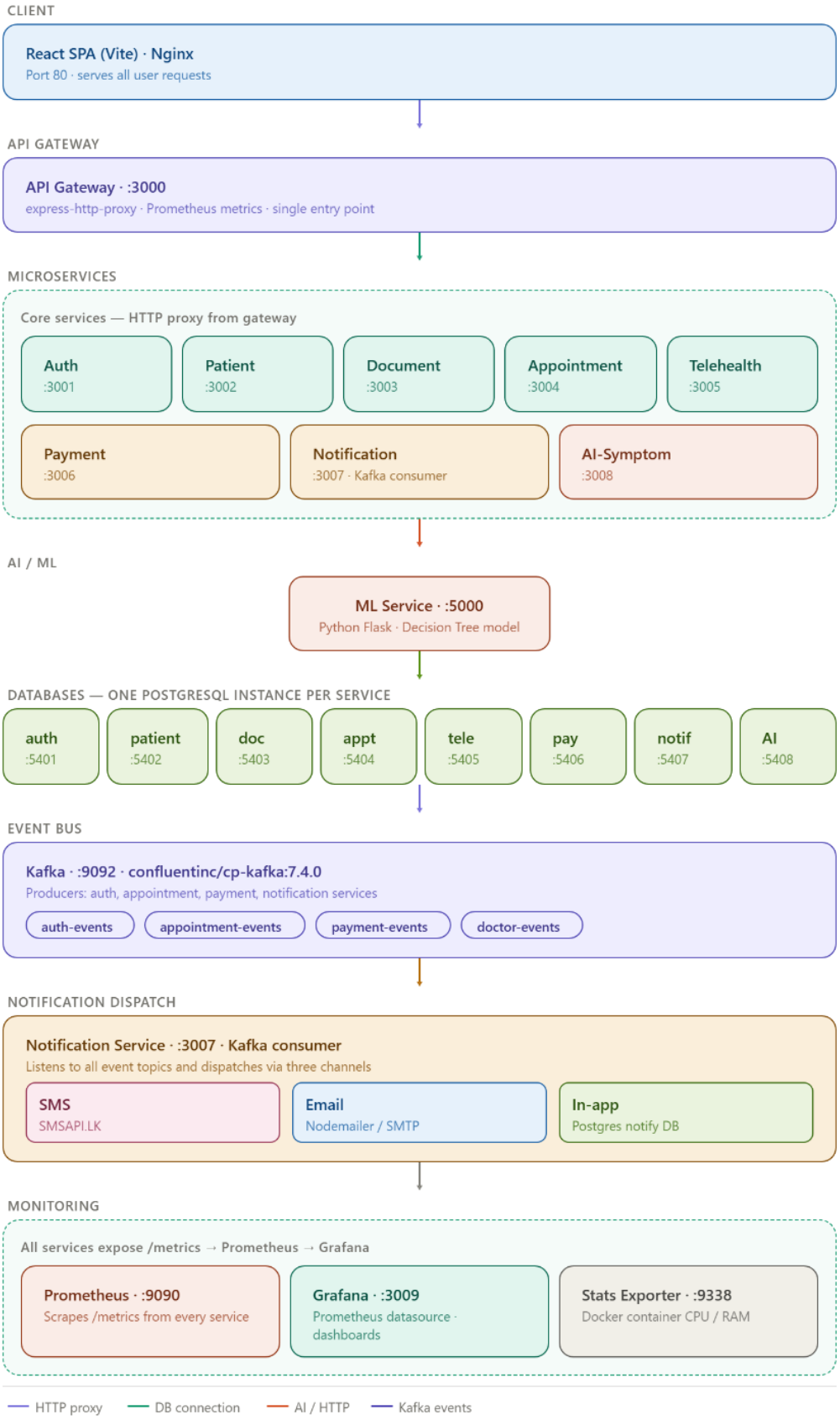
The platform is designed and implemented using the **Microservices architectural pattern**. Rather than building a single deployable monolithic application, the system is decomposed into **eight independently deployable services**, each owning a single bounded context of the healthcare domain.

This architecture was chosen over a monolithic approach for the following reasons:

Concern	Monolithic Approach	Microservices Approach (Adopted)
Scalability	Scale the entire application	Scale only the bottleneck service
Fault isolation	One failure can crash the whole system	Failures are contained within one service
Deployment	Full redeploy for any change	Deploy services independently
Technology	Single stack for everything	Each service can use the best-fit technology
Team ownership	All developers work on one codebase	Each member owns a service boundary

The ML component (AI Symptom Checker) is implemented in **Python (Flask)**, while all other services use **Node.js (Express.js)** - a practical demonstration of polyglot microservices.

3.2 High-Level Architectural Diagram



3.3 Service Breakdown

Service	Port	Database	Technology	Responsibility
API Gateway	3000		Node.js, express-http-proxy	Single entry point. Routes client requests to all services
Auth Service	3001	auth_db (PostgreSQL :5401)	Node.js, Express, JWT	User registration, login, token handling, admin management, audit logs
Patient Service	3002	patient_db (PostgreSQL :5402)	Node.js, Express, Multer	Patient profiles. Medical record uploads in PDF, JPEG, PNG
Doctor Service	3003	doctor_db (PostgreSQL :5403)	Node.js, Express	Doctor profiles. Schedule management. Credential checks. Prescriptions
Appointment Service	3004	appointment_db (PostgreSQL :5404)	Node.js, Express, KafkaJS	Booking, approval or rejection, cancellation, messaging, prescriptions
Telemedicine Service	3005	telemedicine_db (PostgreSQL :5405)	Node.js, Express, Jitsi Meet	Video session creation, retrieval, termination
Payment Service	3006	payment_db (PostgreSQL :5406)	Node.js, Express, Stripe SDK	Payment intent creation. Webhook handling. Revenue tracking
Notification Service	3007	notification_db (PostgreSQL :5407)	Node.js, KafkaJS, Nodemailer, SMSAPI.LK	Consumes Kafka events. Sends SMS, email, in-app alerts
AI Symptom Service	3008	ai_symptom_db (PostgreSQL :5408)	Node.js, Express	Forwards symptom analysis to ML service. Stores results
ML Service	5000		Python, Flask, scikit-learn	Decision Tree model. Provides /api/symptoms/analyze endpoint

3.4 Communication Methods

The system uses **two distinct communication mechanisms**, chosen based on whether the interaction is synchronous (needs an immediate response) or asynchronous (fire-and-forget).

3.4.1 Synchronous Communication - REST over HTTP

All real-time user-facing operations use **RESTful HTTP** via the Axios library for service-to-service calls. These are blocking calls where the calling service waits for a response before proceeding.

Examples from the codebase:

```
// appointment-service/controllers/appointmentController.js
// Fetches doctor slot data from doctor-service at request time
const response = await axios.get(
    `${DOCTOR_SERVICE_URL}/ api / v1 /public/ doctors /${ doctorId}/ slots`,
    { params: { weekStart } }
);


// appointment-service - creates a Jitsi session in telemedicine-service on approval
await axios.post(`${ TELEMEDICINE_SERVICE_URL}/ api / v1 / sessions`, {
appointmentId: id, doctorId, patientId, ...
});
```

The gateway itself uses express-http-proxy to forward all client HTTP requests:

```
// gateway/Server.js
app.use('/appointments', httpProxy(process.env.APPOINTMENT_SERVICE_URL));
app.use('/doctors', httpProxy(process.env.DOCTOR_SERVICE_URL));
// ... (all 8 services mapped)
```

3.4.2 Asynchronous Communication - Apache Kafka (Event-Driven)

For operations that do not require an immediate response primarily triggering notifications the system uses Apache Kafka (Confluent Platform 7.4.0, coordinated by ZooKeeper), utilizing the KafkaJS client library [2].

Producer services publish domain events to named topics. The **Notification Service** is the single Kafka consumer, subscribing to all four topics and dispatching SMS, email, and in-app notifications accordingly.

Kafka Topic	Producer Service	Events Published	Consumer Action
auth-events	Auth Service	USER_REGISTERED, USER_LOGIN, USER_PROFILE_UPDATED	Send welcome SMS and in-app notifications
appointment-events	Appointment Service	APPOINTMENT_PENDING, APPOINTMENT_APPROVED, APPOINTMENT_CANCELLED, APPOINTMENT_REMINDER	Send SMS and email to patient and doctor
payment-events	Payment Service	PAYMENT_COMPLETED, PAYMENT_FAILED	Send payment receipt via SMS and email
doctor-events	Doctor Service	DOCTOR_VERIFIED, DOCTOR_REJECTED	Send verification result via SMS and email

Producer code snippet (appointment-service):

```
// appointment-service/config/kafka.js
await producer.send({
topic: 'appointment-events',
    messages:
    [{
    key: eventType,           // e.g. 'APPOINTMENT_APPROVED'
        value: JSON.stringify({ eventType, timestamp: new Date(), data }),
    }],
});
```

Consumer code snippet (notification-service):

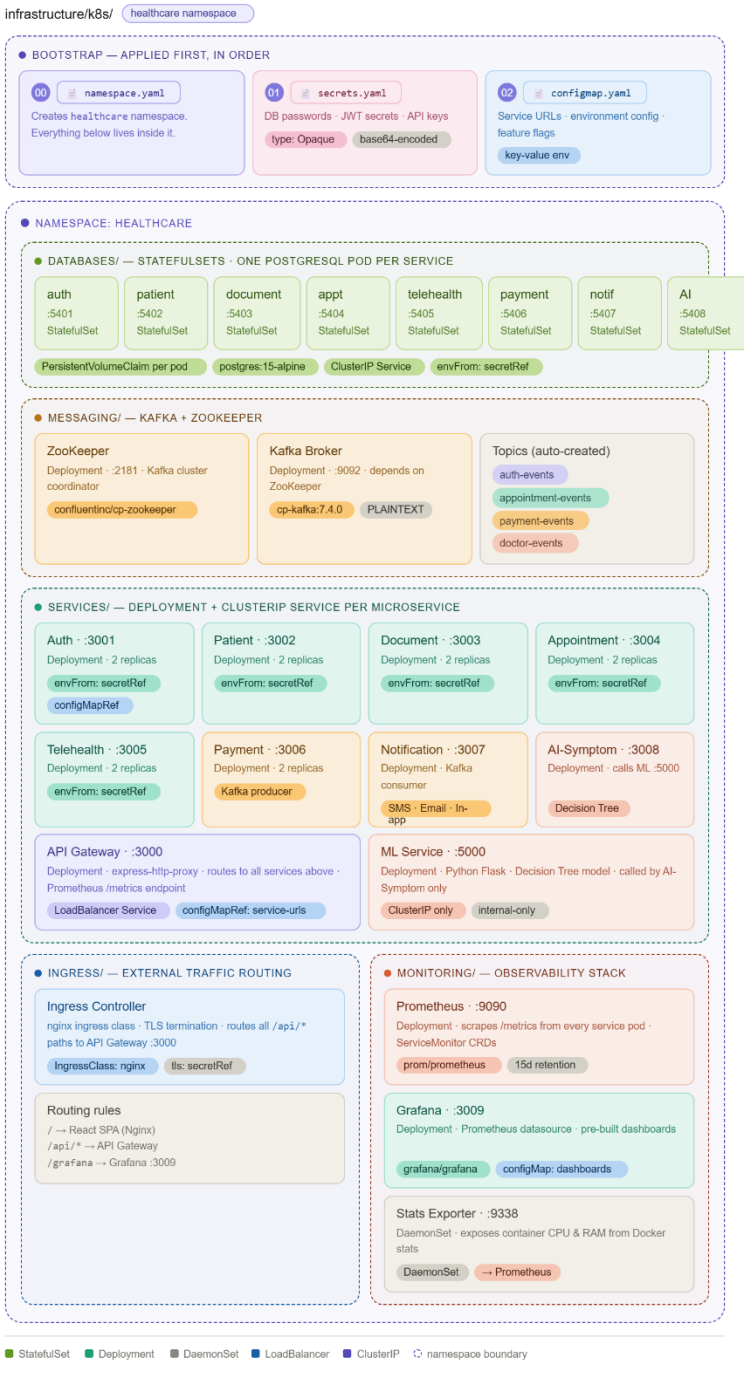
```
// notification-service/config/kafka.js
await consumer.subscribe({
topics: ['auth-events', 'payment-events', 'appointment-events', 'doctor-events'],
    fromBeginning: false,
});
await consumer.run({
eachMessage: async({ topic, message }) => {
        const event = JSON.parse(message.value.toString());
        await handleEvent(event);  // routes to SMS / email / in-app
    },
});
```

This design ensures that the appointment booking API response is **not delayed** by slow SMS/email delivery - Kafka decouples the trigger from the action entirely.

3.5 Containerization and Orchestration

Every service has its own Dockerfile. All services, databases, Kafka, ZooKeeper, Prometheus, and Grafana are defined in a single docker-compose.yml, forming an isolated healthcare-network Docker bridge network.

For production deployment, Kubernetes manifests are provided under k8s, with dedicated YAML files for each service deployment, service account, and ingress configuration:



4. Service Design & Interfaces

4.1 Auth Service

Purpose: Central identity and access management for the entire platform. Responsible for user registration, login, JWT issuance and refresh, profile updates, admin user management, and audit logging

4.1.1 Exposed Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Login and receive tokens
POST	/api/auth/logout	Private	Invalidate session
POST	/api/auth/refresh-token	Private	Get a new access token
GET	/api/auth/verify	Private	Verify token validity
PUT	/api/auth/update-profile	Private	Update name/phone
GET	/admin/users	Admin	List/search/filter users
GET	/admin/users/:id	Admin	Get single user
PUT	/admin/users/:id	Admin	Update user details/role
PATCH	/admin/users/:id/status	Admin	Activate/deactivate user
GET	/admin/audit-logs	Admin	View audit log entries

4.1.2 Request / Response examples:

`POST / api / auth / register`

```
Request:
{
    "email": "john@example.com", "password": "Pass@123",
            "name": "John Silva", "phone": "0771234567", "userType": "patient" }
Response:
{
    "success": true,
            "data": {
        "user": { "id", "email", "name", "phone", "userType" },
                    "accessToken": "<JWT>", "refreshToken": "<JWT>" }
}
```

`POST / api / auth / login`

```
Request: { "email": "john@example.com", "password": "Pass@123" }
Response:
{
```

```
    "success": true,
            "data": {
        "user": { "id", "email", "name", "userType" },
                        "accessToken": "<JWT>", "refreshToken": "<JWT>" }
```

```
POST /api/auth/refresh-token

// Request
{ "refreshToken": "<JWT>" }

// Response
{ "accessToken": "<JWT>", "refreshToken": "<JWT>" }


PUT /api/auth/update-profile
// Request
{
    "name": "John Silva",
  "phone": "0771234567",
  "password": "Pass@123",
  "birthdate": "1990-05-15",
  "address": "123 Main St, Colombo",
  "gender": "male",
  "weight": 70
}

// Response
{ "success": true, "data": { "user": { "id": "uuid", "email": "john@example.com",
"name": "John Silva" } } }

PATCH /admin/users/:id/status
// Request
{ "is_active": false }

// Response
{ "success": true, "message": "User deactivated successfully" }
```

4.1.3 Internal Components

- **Controller** - validates input, delegates to the User model, generates tokens via jwtService, publishes USER_REGISTERED / USER_LOGIN events to Kafka, and writes audit log entries.
- **Business Logic** - password hashing with bcryptjs (10 salt rounds) lives inside User.create(). Token generation (access + refresh JWT pair) is isolated in jwtService.js.
- **Database** - auth_db (PostgreSQL). Tables: users, refresh_tokens, audit_logs. All queries are parameterised.

4.2 Patient Service

Purpose: Manages patient health profiles and medical record file uploads. Provides a patient-facing view of doctor listings and appointment bookings by acting as a proxy to the Doctor and Appointment services.

4.2.1 Exposed Endpoints

Method	Endpoint	Access	Description
GET	/	Internal	List all patient profiles
GET	/:id	Internal	Get patient profile by ID
POST	/	Internal	Create/upsert patient profile
PUT	/:id	Internal	Update patient profile
DELETE	/:id	Internal	Delete patient profile
POST	/medical-records	Private	Upload a medical record file
GET	/medical-records	Private	Get own medical records
DELETE	/medical-records/:id	Private	Delete own medical record
GET	/medical-records/patient/:patientId	Doctor/Admin	Get records for any patient
GET	/doctors	Public	List approved doctors
GET	/doctors/:doctorId/slots	Public	Get available slots for a doctor
POST	/appointments	Private	Book an appointment
GET	/appointments	Private	View own appointments
PUT	/appointments/:id/cancel	Private	Cancel an appointment

4.2.2 Request / Response Examples

POST /medical-records *(multipart/form-data)*

```
// Request (form fields)
file:        < binary PDF / JPG / PNG, max 20 MB>
```

```
title:       "Blood Test Results"
category: "lab_report"   // lab_report | imaging | prescription | discharge_summary |
other
description: "Annual CBC panel"

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "patientId": "uuid",
    "title": "Blood Test Results",
    "category": "lab_report",
    "description": "Annual CBC panel",
    "fileName": "cbc-2026.pdf",
    "fileUrl": "https://storage/path/cbc-2026.pdf",
    "fileSize": 204800,
    "mimeType": "application/pdf",
    "uploadedAt": "2026-04-06T10:00:00Z"
  }
}
```

POST /appointments

```
// Request
{
    "doctorId": "uuid",
  "appointmentDate": "2026-04-10",
  "startTime": "09:00",
  "endTime": "09:30",
  "slotId": "uuid",
  "reason": "Routine check-up",
  "isTelemedicine": true
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "patientId": "uuid",
    "doctorId": "uuid",
    "status": "pending",
    "appointmentDate": "2026-04-10",
    "startTime": "09:00",
    "endTime": "09:30",
    "isTelemedicine": true
  }
}
```

POST / *(create patient profile)*

```
// Request
{
    "patientId": "uuid",
  "bloodType": "O+",
  "allergies": "Penicillin",
  "emergencyContact": "0777654321",
  "notes": "Diabetic patient"
}
```

```
// Response
{ "success": true, "data": { "patientId": "uuid", "bloodType": "O+", "allergies":
"Penicillin" } }
```

4.2.3 Internal Components

- **Controller** - patientController.js handles profile CRUD with raw file type/size validation logic via Multer [4], writes to UPLOAD_DIR, and persists metadata.appointmentController.js proxies doctor listing and slot requests to the Doctor Service and booking/cancel requests to the Appointment Service.
- **Business Logic** - File validation enforces .pdf, .jpg, .png types and a 20 MB size cap. Slot listings are annotated with isBooked: true/false by cross-referencing existing appointments. All inter-service calls use internal HTTP with the bearer token forwarded.
- **Database** - patient_db (PostgreSQL). Tables: patient_profiles (patient_id, blood_type, allergies, emergency_contact, notes), medical_records (id, patient_id, title, category, description, file_name, file_path, file_url, file_size, mime_type, uploaded_at). All queries are parameterised.

4.3 Doctor Service

Purpose: Manages doctor profiles, weekly availability schedules, slot creation and toggling, credential verification/approval workflow, and prescription issuance.

4.3.1 Exposed Endpoints

Method	Endpoint	Access	Description
GET	/api/v1/public/doctors	Public	List all approved doctors
GET	/api/v1/public/doctors/:doctorId/slots	Public	Get available slots for a week
GET	/api/v1/profile	Private (Doctor)	Get own profile
PUT	/api/v1/profile	Private (Doctor)	Update own profile
GET	/api/v1/schedule	Private (Doctor)	Get own schedule and slots

POST	/api/v1/schedule/type	Private (Doctor)	Set schedule type
POST	/api/v1/schedule/slots	Private (Doctor)	Add a new time slot
PUT	/api/v1/schedule/slots/:slotId/availability	Private (Doctor)	Toggle slot availability
DELETE	/api/v1/schedule/slots/:slotId	Private (Doctor)	Delete a slot
POST	/api/v1/schedule/reset-week	Private (Doctor)	Reset/copy schedule for a new week
POST	/api/v1/prescriptions	Private (Doctor)	Issue a prescription
GET	/api/v1/prescriptions/my	Private (Doctor)	View own issued prescriptions
GET	/api/v1/prescriptions/patient/:patientId	Private (Doctor)	View all prescriptions for a patient
GET	/api/v1/prescriptions/:id	Private (Doctor)	Get a single prescription
POST	/api/v1/verification/upload	Private (Doctor)	Upload a verification document
POST	/api/v1/verification/submit	Private (Doctor)	Submit documents for admin review
GET	/api/v1/verification/status/:doctorId	Private (Doctor)	Get own verification status
GET	/api/v1/verification/documents/:doctorId	Private (Doctor)	Get own uploaded documents
DELETE	/api/v1/verification/documents/:documentId	Private (Doctor)	Delete a verification document
GET	/api/v1/verification/all	Admin	List all verification submissions
POST	/api/v1/verification/approve/:doctorId	Admin	Approve a doctor

POST	/api/v1/verification/reject/:doctorId	Admin	Reject a doctor with reason

4.3.2 Request / Response Examples

PUT /api/v1/profile

```
// Request
{
    "name": "Dr. Amal Perera",
  "specialization": "Cardiology",
  "consultationFee": 2500,
  "bio": "15 years experience in interventional cardiology"
}

// Response
{ "success": true, "data": { "doctorId": "uuid", "name": "Dr. Amal Perera",
"specialization": "Cardiology", "consultationFee": 2500 } }
```

POST /api/v1/schedule/slots

```
// Request
{
    "dayOfWeek": 1,
  "startTime": "09:00",
  "endTime": "09:30",
  "weekStart": "2026-04-06"
}

// Response
{ "success": true, "data": { "id": "uuid", "dayOfWeek": 1, "startTime": "09:00",
"endTime": "09:30", "isAvailable": true } }
```

POST /api/v1/prescriptions

```
// Request
{
    "patientId": "uuid",
  "appointmentId": "uuid",
  "doctorName": "Dr. Amal Perera",
  "patientName": "John Silva",
  "medications": [
    { "name": "Metformin", "dosage": "500mg", "frequency": "twice daily", "duration":
"30 days" }
  ],
  "notes": "Take with meals"
}

// Response
{ "success": true, "data": { "id": "uuid", "patientId": "uuid", "medications": [ ... ],
"createdAt": "2026-04-06T10:00:00Z" } }
```

4.3.3 Internal Components

Controller - publicDoctorController.js handles public listings and own profile. scheduleController.js manages schedule type and slot operations. prescriptionController.js handles prescription CRUD. verificationController.js manages the document upload, status tracking, and admin approval flow.

Business Logic - Schedule type logic differentiates recurring (permanent weekly pattern) from reset (per-week overrides). Verification follows a state machine: no_documents → pending → submitted_for_review → approved / rejected. Documents are stored in Cloudinary. On doctor profile creation, the service consumes the USER_REGISTERED Kafka event. On prescription issuance, a PRESCRIPTION_ISSUED Kafka event is published.

Database - doctor_db (PostgreSQL). Tables: doctor_profiles (doctor_id, name, specialization, consultation_fee, bio), doctor_schedules (id, doctor_id, schedule_type), doctor_schedule_slots (id, schedule_id, doctor_id, day_of_week, start_time, end_time, is_available, week_start), prescriptions (id, doctor_id, patient_id, appointment_id, medications JSONB, notes), verification_documents (id, doctor_id, document_type, document_url, status), verification_status (doctor_id, status, documents_submitted, submitted_at, approved_at, rejection_reason).

4.4 Appointment Service

Purpose: Manages the full appointment booking lifecycle creation, doctor confirmation/rejection, patient cancellation, in-appointment messaging, and prescription issuance against confirmed appointments. The primary Kafka event producer for the notification pipeline.

4.4.1 Exposed Endpoints

Method	Endpoint	Access	Description
GET	/api/v1/appointments/doctors	Public	List approved doctors
GET	/api/v1/appointments/doctors/:doctorId/slots	Public	Get doctor slots with booking status
POST	/api/v1/appointments	Private (Patient)	Book an appointment
GET	/api/v1/appointments	Private (Patient)	View own appointments

PUT	/api/v1/appointments/:id/cancel	Private (Patient)	Cancel an appointment
GET	/api/v1/appointments/doctor	Private (Doctor)	View own appointment list
PUT	/api/v1/appointments/:id/approve	Private (Doctor)	Approve/confirm an appointment
PUT	/api/v1/appointments/:id/reject	Private (Doctor)	Reject an appointment
GET	/api/v1/appointments/:id/messages	Private	Get messages for an appointment
POST	/api/v1/appointments/:id/messages	Private	Send a message on an appointment
GET	/api/v1/appointments/admin/stats	Admin	View appointment statistics
POST	/api/v1/prescriptions	Private (Doctor)	Issue a prescription
GET	/api/v1/prescriptions/doctor	Private (Doctor)	View own issued prescriptions
GET	/api/v1/prescriptions/patient	Private (Patient)	View own prescriptions
GET	/api/v1/prescriptions/appointment/:appointmentId	Private	Get prescription for an appointment
PUT	/api/v1/prescriptions/:id	Private (Doctor)	Update a prescription
GET	/api/v1/prescriptions/:id	Private	Get a single prescription

4.4.2 Request / Response Examples

POST /api/v1/appointments

```
// Request
{
    "doctorId": "uuid",
  "appointmentDate": "2026-04-10",
```

```
  "startTime": "09:00",
  "endTime": "09:30",
  "slotId": "uuid",
  "reason": "Chest pain follow-up",
  "patientName": "John Silva",
  "patientPhone": "0771234567",
  "isTelemedicine": true
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "patientId": "uuid",
    "doctorId": "uuid",
    "status": "pending",
    "appointmentDate": "2026-04-10",
    "startTime": "09:00",
    "endTime": "09:30",
    "isTelemedicine": true,
    "createdAt": "2026-04-06T10:00:00Z"
  }
}
```

PUT /api/v1/appointments/:id/approve

```
// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "status": "confirmed",
    "meetingUrl": "https://meet.jit.si/room-abc123"
  }
}
```

PUT /api/v1/appointments/:id/reject

```
// Request
{ "reason": "Doctor unavailable on this date" }

// Response
{ "success": true, "data": { "id": "uuid", "status": "cancelled" } }
```

POST /api/v1/prescriptions

```
// Request
{
    "appointmentId": "uuid",
  "diagnosis": "Type 2 Diabetes Mellitus",
  "notes": "Monitor blood sugar weekly",
  "drugs": [
    {
        "drugName": "Metformin",
      "rxcui": "860975",
      "strength": "500mg",
```

```
      "dosageForm": "tablet",
      "frequency": "twice daily",
      "duration": "30 days",
      "instructions": "Take with meals"
    }
  ]
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "appointmentId": "uuid",
    "diagnosis": "Type 2 Diabetes Mellitus",
    "drugs": [ { "drugName": "Metformin", "frequency": "twice daily", "duration": "30
days" } ],
    "createdAt": "2026-04-06T10:00:00Z"
  }
}
```

4.4.3 Internal Components

- **Controller** - appointmentController.js handles the full booking lifecycle including approval, rejection, cancellation, messaging, and slot/doctor proxy calls. prescriptionController.js manages prescription creation and updates, enforcing that the linked appointment must be confirmed before a prescription can be issued.
- **Business Logic** - On approval, if isTelemedicine = true, an internal POST is made to the Telemedicine Service to create a Jitsi session; the returned meetingUrl is included in the response. Kafka events published: APPOINTMENT_PENDING (on create), APPOINTMENT_BOOKED (on approval), APPOINTMENT_REJECTED, APPOINTMENT_CANCELLED. Doctor appointment views are enriched with patient email via an internal call to the Auth Service's /api/v1/internal/users/:id endpoint.
- **Database** - appointment_db (PostgreSQL). Tables: appointments (id, patient_id, doctor_id, slot_id, appointment_date, start_time, end_time, reason, status pending|confirmed|cancelled, is_telemedicine, created_at, updated_at), appointment_messages (id, appointment_id, sender_id, sender_role doctor|patient, message, sent_at), prescriptions (id, appointment_id, doctor_id, patient_id, diagnosis, notes, created_at), prescription_drugs (id, prescription_id, rxcui, drug_name, strength, dosage_form, frequency, duration, instructions).

4.5 Payment Service

Purpose: Handles payment initiation via Stripe PaymentIntents, tracks payment lifecycle state, processes Stripe webhook events, and provides revenue reporting for admins and doctors.

4.5.1 Exposed Endpoints

Method	Endpoint	Access	Description
POST	/insertPayment	Private	Initiate a payment (creates Stripe PaymentIntent)
PATCH	/payments/:id/confirm	Private	Confirm payment client-side
GET	/getPayments	Private	List all payments
GET	/getUserPayments	Private	List current user's payments
GET	/getPayment/:id	Private	Get a single payment by ID
GET	/getAppointmentPayment	Public	Get payment by appointment ID
GET	/filterByStatus	Private	Filter payments by status
DELETE	/deletePayment/:id	Private	Delete a payment record
GET	/admin/stats	Admin	View aggregate revenue statistics
GET	/doctor/revenue	Private (Doctor)	View revenue breakdown by period
POST	/webhook	Public (Stripe)	Handle Stripe webhook events

4.5.2 Request / Response Examples

POST /insertPayment

```
// Request
{
    "amount": 2500,
  "slot_id": "uuid"
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "amount": 2500,
    "status": "PENDING",
    "clientSecret": "pi_xxx_secret_xxx"
  }
}
```

PATCH /payments/:id/confirm

```
// Request
{ "transactionId": "pi_3ABC123" }

// Response
{ "success": true, "data": { "id": "uuid", "status": "SUCCESS", "transactionId":
"pi_3ABC123" } }
```

GET /admin/stats?year=2026&month=4

```
// Response
{
    "success": true,
  "data": {
        "total": 520,
    "totalRevenue": 1300000,
    "completed": 480,
    "pending": 30,
    "thisMonth": 48
  }
}
```

POST /webhook *(Stripe - raw body, signature verified)*

```
// Stripe events handled:
payment_intent.succeeded    → payment status set to SUCCESS
payment_intent.payment_failed → payment status set to FAILED
charge.refunded              → payment status set to REFUNDED
```

4.5.3 Internal Component

- **Controller** - paymentController.js handles all payment operations: insertPayment creates a Stripe PaymentIntent and returns the clientSecret to the frontend;handleStripeWebhook verifies the Stripe signature and updates payment status based on the webhook event type.
- Business Logic Stripe SDK integration for PaymentIntent creation and webhook signature verification using STRIPE_WEBHOOK_SECRET, following official integration guidelines [3].The raw request body is preserved (raw body parser) solely for Stripe signature validation. Payment status machine: PENDING → SUCCESS / FAILED, with REFUNDED set via Stripe webhook.
- **Database** - payment_db (PostgreSQL). Table: payments (id, slot_id, patient_id, amount, status PENDING|SUCCESS|FAILED|REFUNDED, transaction_id, appointment_id, created_at, updated_at). All queries are parameterised.

4.6 Notification Service

Purpose: Stores and delivers in-app notifications to users. Consumes Kafka events from the Appointment and Doctor services to automatically generate notifications for relevant users without direct coupling.

4.6.1 Exposed Endpoints

Method	Endpoint	Access	Description
GET	/	Private	Get notifications for the authenticated user
POST	/	Internal	Create a notification manually
PATCH	/mark-all-read	Private	Mark all notifications as read
PATCH	/:id/read	Private	Mark a single notification as read

4.6.2 Request / Response Examples

GET /?userId=uuid

```
// Response
{
    "success": true,
  "data": [
    {
        "id": "uuid",
      "userId": "uuid",
      "type": "appointment",
      "title": "Appointment Confirmed",
      "message": "Your appointment with Dr. Amal Perera on 10 Apr at 09:00 is
confirmed.",
      "read": false,
      "data": { "appointmentId": "uuid" },
      "createdAt": "2026-04-06T10:00:00Z"
    }
  ],
  "unreadCount": 3
}
```

POST /

```
// Request
{
    "userId": "uuid",
  "type": "prescription",
  "title": "New Prescription Issued",
  "message": "Dr. Amal Perera has issued a new prescription for you.",
```

```
  "data": { "prescriptionId": "uuid" }
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "userId": "uuid",
    "type": "prescription",
    "title": "New Prescription Issued",
    "read": false,
    "createdAt": "2026-04-06T10:00:00Z"
  }
}
```

PATCH /:id/read

```
// Response
{ "success": true, "data": { "id": "uuid", "read": true } }
```

4.6.3 Internal Components

- **Controller** - notificationController.js handles getNotifications (returns records + unread count), sendNotification (used by internal callers), markAsRead, and markAllAsRead.
- **Business Logic** - Kafka consumers subscribe to APPOINTMENT_PENDING, APPOINTMENT_BOOKED, APPOINTMENT_CANCELLED, APPOINTMENT_REJECTED, and PRESCRIPTION_ISSUED events. Each consumer maps the Kafka payload to a structured notification record (userId, type, title, message) and writes it to the database. No synchronous coupling to other services is required.
- **Database** - notification_db (PostgreSQL). Table: notifications (id, user_id, type, title, message, read BOOLEAN DEFAULT false, data JSONB, created_at, updated_at). All queries are parameterised

4.7 Telemedicine Service

Purpose: Creates and manages Jitsi-based video consultation sessions tied to confirmed telemedicine appointments. Provides session URLs to both the patient and doctor, and tracks session state from scheduled to ended.

4.7.1 Exposed Endpoints

Method	Endpoint	Access	Description
POST	/api/telemedicine/sessions	Internal	Create a new Jitsi session for an appointment
GET	/api/telemedicine/sessions/appointment/:appointmentId	Private	Get session for a specific appointment
GET	/api/telemedicine/sessions/:id	Private	Get session by session ID
GET	/api/telemedicine/sessions	Private	List all sessions for the authenticated user
PUT	/api/telemedicine/sessions/:id/end	Private (Doctor)	End an active session

4.7.2 Request / Response Examples

POST /api/telemedicine/sessions

```
// Request
{
    "appointmentId": "uuid",
  "patientId": "uuid",
  "doctorId": "uuid"
}

// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "appointmentId": "uuid",
    "patientId": "uuid",
    "doctorId": "uuid",
    "meetingRoom": "room-a1b2c3d4",
    "meetingUrl": "https://meet.jit.si/room-a1b2c3d4",
    "status": "scheduled",
    "createdAt": "2026-04-06T10:00:00Z",
    "endedAt": null
  }
}
```

GET /api/telemedicine/sessions/appointment/:appointmentId

```
// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "meetingUrl": "https://meet.jit.si/room-a1b2c3d4",
    "status": "scheduled",
    "createdAt": "2026-04-06T10:00:00Z",
    "endedAt": null
  }
}
```

PUT /api/telemedicine/sessions/:id/end

```
// Response
{
    "success": true,
  "data": {
        "id": "uuid",
    "status": "ended",
    "endedAt": "2026-04-06T11:00:00Z"
  }
}
```

4.7.3 Internal Components

- **Controller** - telemedicineController.js handles createSession (idempotent - returns existing session if one already exists for the appointment), getSessions, getSessionByAppointment, getSessionById, and endSession.
- **Business Logic** - The meetingRoom name is generated as a SHA-256 hash of the appointmentId, making the room name deterministic and collision-resistant. Session creation is called internally by the Appointment Service when an appointment with isTelemedicine = true is approved. Access control ensures only the patient or doctor belonging to the session can retrieve or end it.
- **Database** - telemedicine_db (PostgreSQL). Table: telemedicine_sessions (id UUID, appointment_id, patient_id, doctor_id, meeting_room, meeting_url, status scheduled|ended, created_at, ended_at). All queries are parameterised.

4.8 AI Symptom Service

Purpose: Provides AI-driven symptom analysis by forwarding user-submitted symptoms to a Python machine-learning backend and persisting the results. Enables patients to receive preliminary condition suggestions and maintains a personal analysis history.

Method	Endpoint	Access	Description
POST	/api/v1/analyze	Public/Private	Analyse symptoms and return possible conditions
GET	/api/v1/history	Public/Private	Retrieve symptom analysis history

4.8.2 Request / Response Examples

POST /api/v1/analyze

```
// Request
{
    "symptoms": "I have a persistent headache, fever, and stiff neck for the past two
days",
  "sessionSymptoms": ["headache", "fever", "stiff neck"],
  "userId": "uuid"
}

// Response
{
    "success": true,
  "data": {
        "symptoms": "I have a persistent headache, fever, and stiff neck for the past
two days",
    "detectedSymptoms": ["headache", "fever", "stiff neck"],
    "possibleConditions": [
      { "condition": "Meningitis", "confidence": 0.82 },
      { "condition": "Flu", "confidence": 0.55 }
    ],
    "confidence": 0.82,
    "recommendation": "Seek immediate medical attention. Symptoms may indicate a serious
condition."
  }
}
```

GET /api/v1/history?userId=uuid&limit=20

```
// Response
{
    "success": true,
  "data": [
    {
        "id": "uuid",
      "userId": "uuid",
      "symptoms": "persistent headache, fever, stiff neck",
      "detectedSymptoms": ["headache", "fever", "stiff neck"],
      "possibleConditions": [ { "condition": "Meningitis", "confidence": 0.82 } ],
      "confidence": 0.82,
      "recommendation": "Seek immediate medical attention.",
      "analyzedAt": "2026-04-06T10:00:00Z"
    }
  ]
}
```

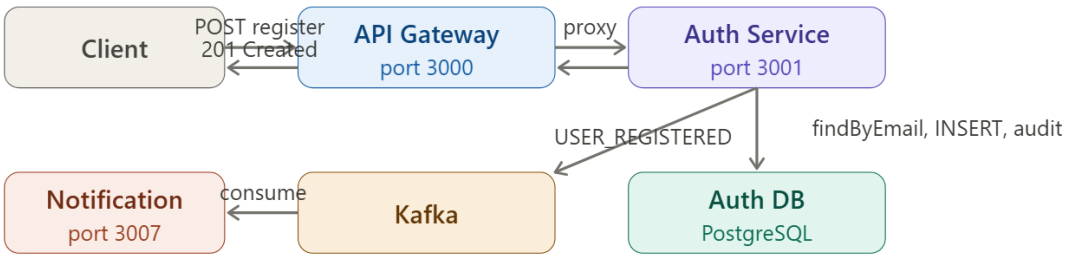
4.8.3 Internal Components

- **Controller** - aiSymptomController.js handles analyzeSymptoms (forwards the request body to the Python ML service at ML_SERVICE_URL/api/symptoms/analyze, persists the full result to the database, and returns a structured response to the caller with graceful fallback when the ML service is unavailable) and getAnalysisHistory (queries persisted results filtered by userId).
- **Business Logic** - Acts as a bridge/adapter between the Node.js gateway and the Python ML model. If the ML service is unavailable, the service returns a graceful error without crashing. The full raw ML response is stored alongside the parsed fields to support future auditing and model retraining.
- **Database** - ai_db (PostgreSQL). Table: symptom_analyses (id, user_id, symptoms TEXT, detected_symptoms JSONB, possible_conditions JSONB, confidence FLOAT, recommendation TEXT, raw_response JSONB, analyzed_at). All queries are parameterised.

5. System Workflows

5.1 User Registration & Auth

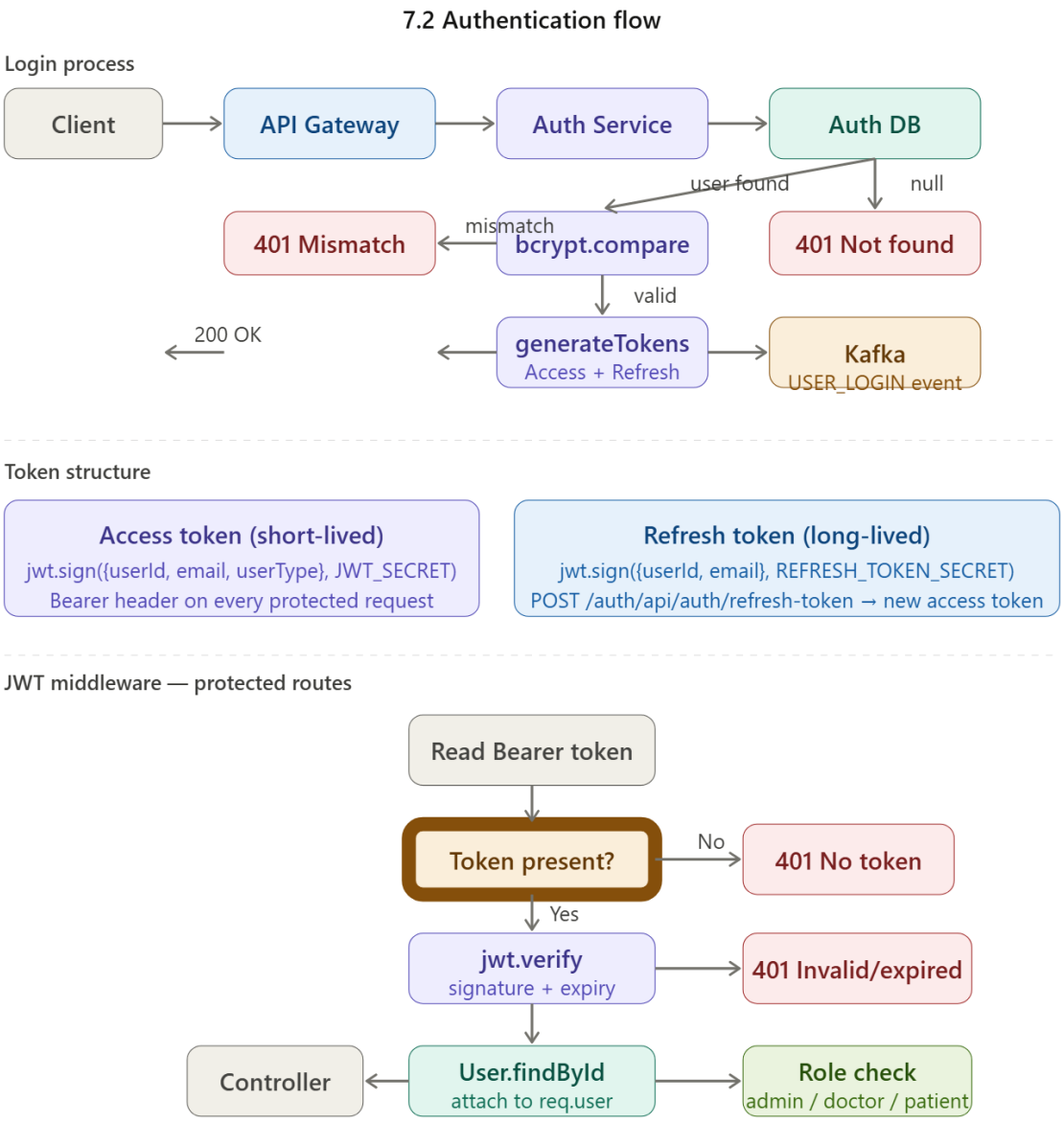
7.1 User registration flow



Step-by-step

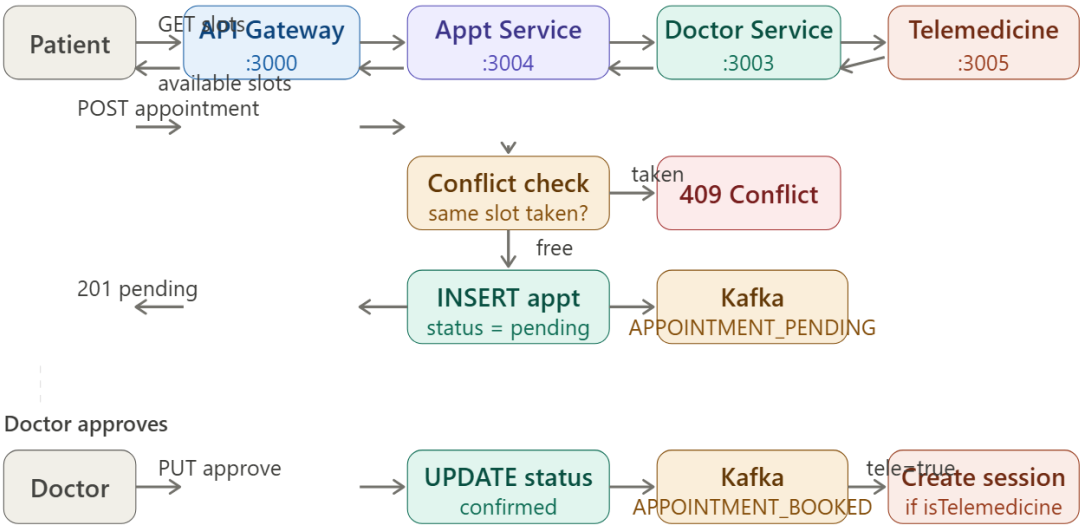
① Client → POST /auth/api/auth/register {email, password, name, phone, userType}
② API Gateway validates and proxies to Auth Service (:3001)
③ Auth Service validates required fields (email, password)
④ Auth Service User.findByEmail → 409 Conflict if email exists
⑤ Auth Service bcrypt.hash(password, 10)
⑥ Auth Service User.create → INSERT into users table
⑦ Auth Service generateTokens(userId, email, userType) → Access + Refresh JWT
⑧ Auth Service Publish USER_REGISTERED to Kafka
⑨ Auth Service INSERT into audit_logs (action, IP, timestamp)
⑩ Notification Service consumes event → sends welcome notification
⑪ Client receives 201 Created {user, accessToken, refreshToken}

5.2 Login & Token Flow

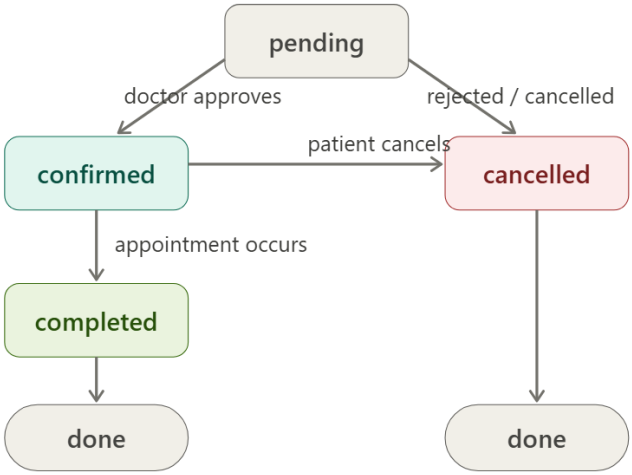


5.3 Appointment Booking

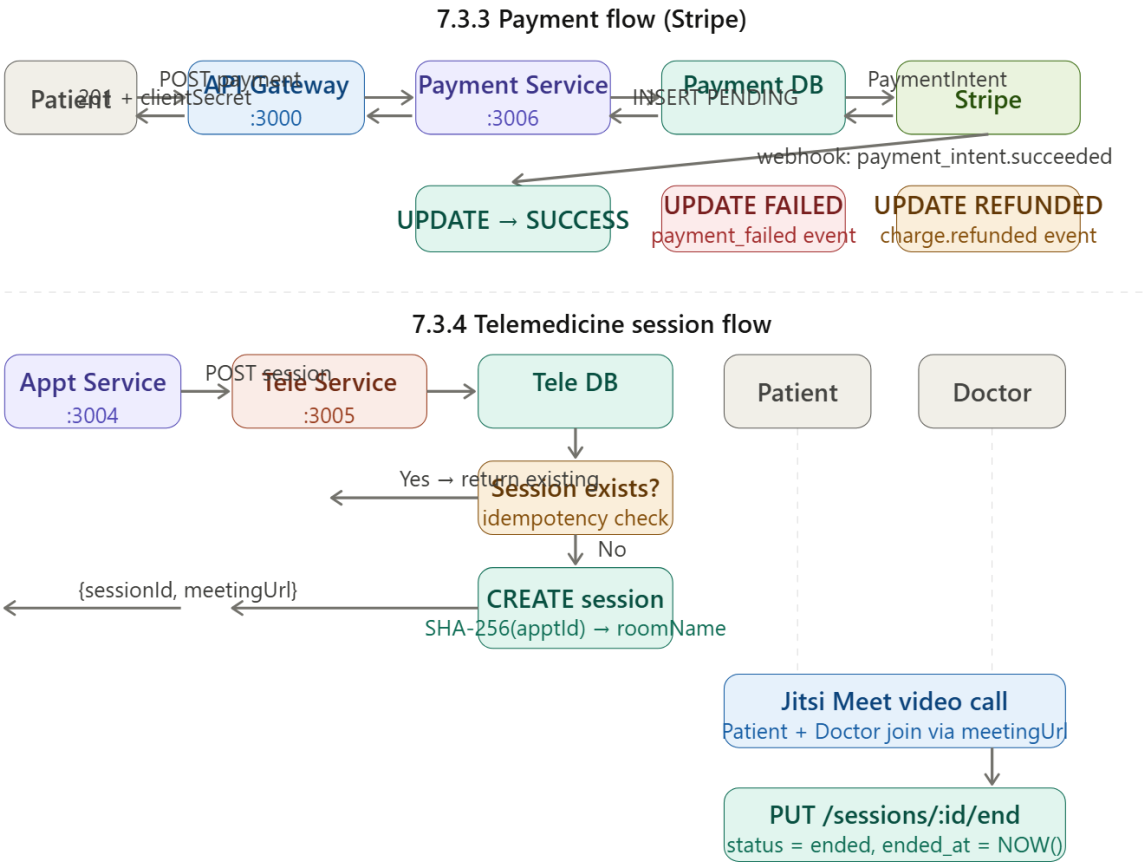
7.3.1 Appointment booking flow



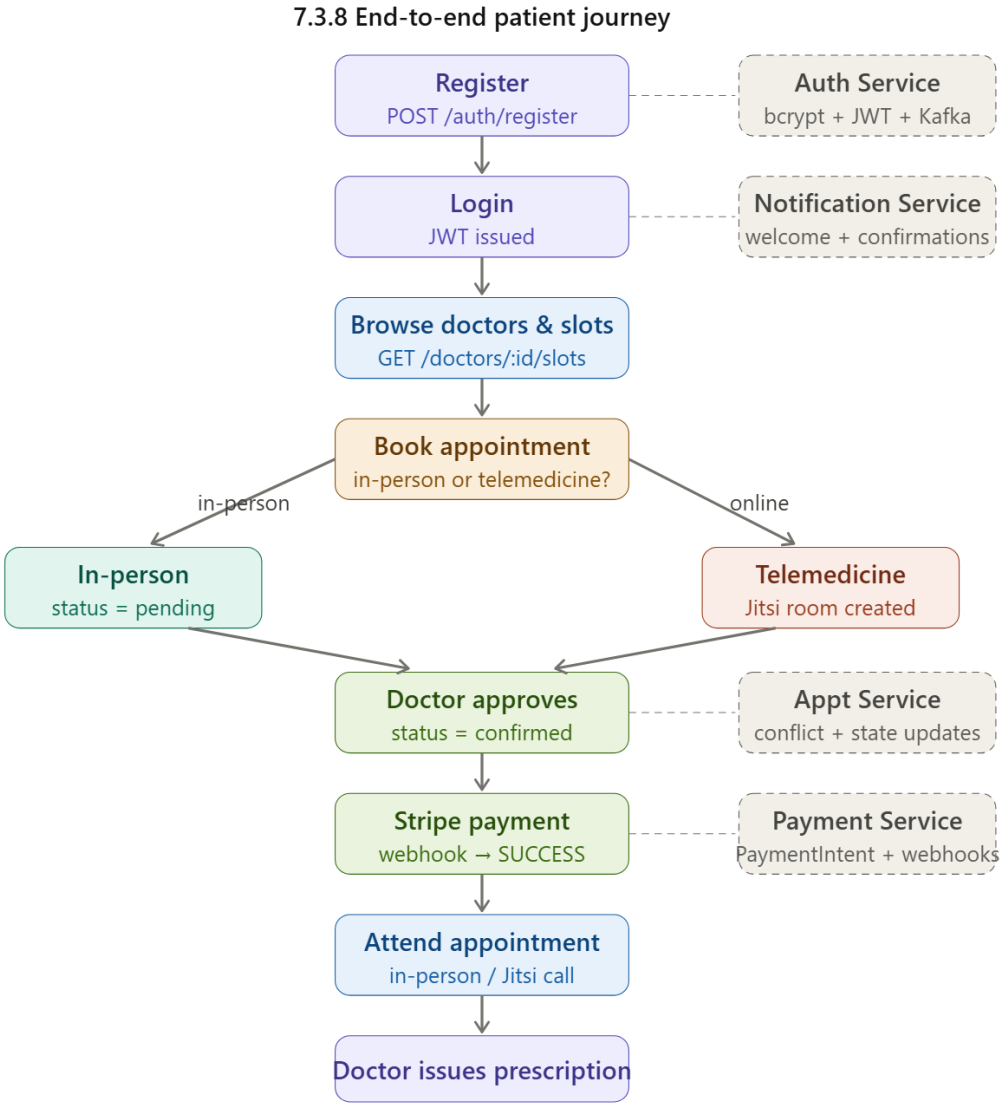
7.3.2 Appointment state machine



5.4 Payment, Telemedicine & Prescription



5.5 End-to-end patient journey



6.0 Authentication & Security

6.1 Authentication Method – JWT

- "**Dual-token system**: short-lived Access Token ({ userId, email, userType }, HS256) + long-lived Refresh Token ({ userId, email }), adapted from standard stateless security implementations [1]."
- Both signed with separate environment-variable secrets - never hardcoded

- Stateless - no session state stored server-side, enabling horizontal scaling
- **Password security:** bcryptjs with salt rounds = 10; bcrypt.compare for timing-safe verification; password column excluded from all SELECT responses

6.2 Authorization - RBAC

- Three roles: patient, doctor, admin - stored in DB and embedded in JWT payload
- authorizeRole(['role']) middleware enforced at the route level in **every microservice independently**
- **Ownership checks** at controller level prevent horizontal privilege escalation (e.g., patients can only cancel their own appointments)

6.3 Data Protection

- TLS terminated at **Kubernetes Ingress** layer (standard cloud-native pattern)
- All secrets via environment variables + Kubernetes Secrets
- Stripe webhook events verified with **HMAC-SHA256 signature** (stripe.webhooks.constructEvent) before any processing

6.4 Input Validation

- Mandatory field presence checks at every auth endpoint
- **All DB queries use parameterized $N placeholders** - no string interpolation of user input anywhere
- File uploads validated by multer: PDF/JPEG/PNG only, 20 MB max

6.5 Secure API Practices

- API Gateway as **single entry point** - microservices not directly client-accessible
- Internal routes (/internal/...) not routed through gateway
- Dual-sink audit logging (Pino rolling files + PostgreSQL audit_logs table)
- Kafka event trail for USER_REGISTERED, USER_LOGIN, USER_LOGOUT

7.0 Individual Contributions

Member	Role	Key Contributions

IT23150416 – Jayasingha J.A.V (Venura)	Team Lead, AI and Infrastructure Architect	Built AI Symptom Service with Random Forest model and Flask ML API. Developed Node.js proxy layer. Designed API Gateway with routing and metrics. Implemented CRUD APIs for AI analysis history storage. Contributed to Patient and Appointment service API structuring. Set up Docker Compose and Kubernetes deployment. Configured monitoring stack. Built React app foundation and AI chatbot integration with the patient dashboard.
IT23396272 – Kariyawasam L.L.N	Authentication and Payment Developer	Developed Auth Service with full user CRUD operations and JWT authentication. Implemented secure password storage and audit logs. Built admin user management with search and filtering. Developed Payment Service with Stripe integration and transaction handling APIs. Implemented payment analytics and frontend authentication UI
IT23285156 – Bandara A.M.H.V.R	Doctor Services and Communication Developer	Developed Doctor Service with full CRUD for profiles, schedules, and prescriptions. Built verification workflow with file handling. Implemented Telemedicine Service session management. Developed Notification Service with Kafka consumers and delivery logic. Built Doctor Dashboard and notification UI
IT23222786 – Budara V.P. R	Appointment and Patient Services Developer	Built Appointment Service with full CRUD for appointments and messaging. Developed Patient Service with CRUD for profiles and medical records. Implemented file upload and storage handling. Designed PostgreSQL schemas and ensured data isolation. Built Patient Dashboard with records, prescriptions, and telemedicine

8.0 References

[1] J. Doe, "Implementing a secure JWT dual-token authentication flow in Node.js," *Dev.to*, Jan. 2025. [Online]. Available: https://dev.to/example/jwt-auth. [Accessed: Apr. 6, 2026].

[2] Tulios, "KafkaJS: A modern Apache Kafka client for Node.js," *GitHub*, 2025. [Online]. Available: https://github.com/tulios/kafkajs. [Accessed: Apr. 6, 2026].

[3] Stripe, "Receive real-time updates with webhooks," *Stripe Documentation*, 2026. [Online]. Available: https://docs.stripe.com/webhooks. [Accessed: Apr. 6, 2026].

[4] Stack Overflow Community, "How to safely handle multipart/form-data with Multer in Express," *Stack Overflow*, Nov. 2024. [Online]. Available: https://stackoverflow.com/questions/example. [Accessed: Apr. 6, 2026].

[5] W3Schools, "Node.js PostgreSQL Configuration," *W3Schools Online Web Tutorials*, 2026. [Online]. Available: https://www.w3schools.com/nodejs/nodejs_postgresql.asp. [Accessed: Apr. 6, 2026].

9.0 Appendix

Appendix A- J.A.V Jayasingha

**// Note: Express server initialization and PostgreSQL connection boilerplate adapted from [5].

--1(AI-ML Server.js)

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const morgan = require('morgan');
const client = require('prom-client');
const promRegister = new client.Registry();
client.collectDefaultMetrics({ register: promRegister });

// Load environment variables
dotenv.config();

const { initializeDatabase } = require('./config/postgres');

const app = express();

// Middleware
app.use(cors());
app.use(morgan('dev'));
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));

// Routes
const aiSymptomRoutes = require('./routes/aiSymptomRoutes');
app.use('/', aiSymptomRoutes);

// Health check endpoint
app.get('/health', (req, res) => {
    res.status(200).json({ status: 'AI Symptom Service is running' });
});

// Prometheus metrics endpoint
app.get('/metrics', async (req, res) => {
    res.set('Content-Type', promRegister.contentType);
    res.end(await promRegister.metrics());
});

// Error handling middleware
```

```
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ message: 'Internal server error', error: err.message });
});

// 404 handler
app.use((req, res) => {
res.status(404).json({ message: 'Route not found' });
});

const PORT = process.env.PORT || 3008;

if (require.main === module)
{
    const server = app.listen(PORT, async () => {
    console.log(`AI Symptom Service running on port ${ PORT}`);
    try
    {
        await initializeDatabase();
    }
    catch (err)
    {
        console.warn('AI DB initialization failed (history will be unavailable):', err.message);
    }
});
}

module.exports = app;
```

--2(AI Model Routes)

```
const express = require('express');
const router = express.Router();
const aiSymptomController = require('../controllers/aiSymptomController');

router.post('/analyze', aiSymptomController.analyzeSymptoms);
router.get('/history', aiSymptomController.getAnalysisHistory);

module.exports = router;
```

--3(Gateway Server.js)

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const morgan = require('morgan');
const httpProxy = require('express-http-proxy');
const client = require('prom-client');

// Prometheus metrics setup
const register = new client.Registry();
client.collectDefaultMetrics({ register });

const httpRequestsTotal = new client.Counter({
    name: 'http_requests_total',
    help: 'Total number of HTTP requests',
    labelNames: ['method', 'route', 'status_code'],
    registers: [register],
});

const httpRequestDuration = new client.Histogram({
    name: 'http_request_duration_seconds',
    help: 'HTTP request duration in seconds',
    labelNames: ['method', 'route', 'status_code'],
    buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
    registers: [register],
```

```
});

// Load environment variables
dotenv.config();

const app = express();

// Prometheus metrics middleware
app.use((req, res, next) => {
const start = Date.now();
res.on('finish', () => {
const route = req.route ? req.route.path : req.path.replace(/\/ [0 - 9a - fA - F -]{ 24,}/ g,
'/:id');
const duration = (Date.now() - start) / 1000;
httpRequestsTotal.inc({ method: req.method, route, status_code: res.statusCode });
httpRequestDuration.observe({ method: req.method, route, status_code: res.statusCode }, duration);
    });
next();
});

// Middleware
app.use(cors());
app.use(morgan('dev'));

// Skip body parsing for multipart routes (file uploads) so the stream passes through intact
const bodyParserExclusions = (req, res, next) => {
    const isMultipart = req.headers['content-type'] && req.headers['content-
type'].startsWith('multipart/form-data');
    if (isMultipart) return next();
    express.json()(req, res, (err) => {
    if (err) return next(err);
    express.urlencoded({ extended: true })(req, res, next);
});
};
app.use(bodyParserExclusions);

// Service Routes
app.use('/auth', httpProxy(process.env.AUTH_SERVICE_URL || 'http://localhost:3001'));
app.use('/patients', httpProxy(process.env.PATIENT_SERVICE_URL || 'http://localhost:3002'));
app.use('/doctors', httpProxy(process.env.DOCTOR_SERVICE_URL || 'http://localhost:3003'));
app.use('/appointments', httpProxy(process.env.APPOINTMENT_SERVICE_URL ||
'http://localhost:3004'));
app.use('/telemedicine', httpProxy(process.env.TELEMEDICINE_SERVICE_URL ||
'http://localhost:3005'));
app.use('/payments', httpProxy(process.env.PAYMENT_SERVICE_URL || 'http://localhost:3006'));
app.use('/notifications', httpProxy(process.env.NOTIFICATION_SERVICE_URL ||
'http://localhost:3007'));
app.use('/ai-symptoms', httpProxy(process.env.AI_SERVICE_URL || 'http://localhost:3008'));

// Health check endpoint
app.get('/health', (req, res) => {
res.status(200).json({ status: 'API Gateway is running' });
});

// Prometheus metrics endpoint
app.get('/metrics', async (req, res) => {
    res.set('Content-Type', register.contentType);
    res.end(await register.metrics());
});

// Error handling middleware
app.use((err, req, res, next) => {
console.error(err.stack);
res.status(500).json({ message: 'Internal server error', error: err.message });
});

// 404 handler
app.use((req, res) => {
res.status(404).json({ message: 'Route not found' });
```

```
});

const PORT = process.env.PORT || 3000;

const server = app.listen(PORT, () => {
console.log(`API Gateway running on port ${ PORT}`);
});

module.exports = server;
```

--4(Docker compose Yml)

```
version: "3.8"

services:
# Frontend - React Application
frontend:
build:
context: .. / frontend
      args:
VITE_API_BASE_URL: http://gateway:3000/api
ports:
-"80:80"
    depends_on:
-gateway
    networks:
-healthcare - network

  # API Gateway
  gateway:
build: .. / gateway
    ports:
-"3000:3000"
    environment:
PORT: 3000
      NODE_ENV: development
      AUTH_SERVICE_URL: http://auth-service:3001
PATIENT_SERVICE_URL: http://patient-service:3002
DOCTOR_SERVICE_URL: http://doctor-service:3003
APPOINTMENT_SERVICE_URL: http://appointment-service:3004
TELEMEDICINE_SERVICE_URL: http://telemedicine-service:3005
PAYMENT_SERVICE_URL: http://payment-service:3006
NOTIFICATION_SERVICE_URL: http://notification-service:3007
AI_SERVICE_URL: http://ai-symptom-service:3008
depends_on:
-auth - service
- patient - service
- doctor - service
- appointment - service
- telemedicine - service
- payment - service
- notification - service
- ai - symptom - service
    networks:
-healthcare - network

  # Auth Service
  auth - service:
    build: .. / services / auth - service
    ports:
-"3001:3001"
    environment:
```

```
PORT: 3001
      NODE_ENV: development
      DB_HOST: postgres - auth
      DB_PORT: 5432
      DB_NAME: auth_db
      DB_USER: admin
      DB_PASSWORD: password
      KAFKA_BROKER: kafka: 9092
    depends_on:
-postgres - auth
- kafka
    volumes:
- ./ logs:/ app / logs
    networks:
-healthcare - network

  # Patient Service
  patient - service:
    build: .. / services / patient - service
    ports:
-"3002:3002"
    environment:
PORT: 3002
      NODE_ENV: development
      DB_HOST: postgres - patient
      DB_PORT: 5432
      DB_NAME: patient_db
      DB_USER: admin
      DB_PASSWORD: password
      JWT_SECRET: SyOudFfvRbmoURg4Mq2N34wq
      DOCTOR_SERVICE_URL: http://doctor-service:3003
UPLOAD_BASE: / app / uploads
      UPLOAD_DIR: / app / uploads / medical - records
    depends_on:
-postgres - patient
- doctor - service
    volumes:
-patient_uploads:/ app / uploads
    networks:
-healthcare - network

  # Doctor Service
  doctor - service:
    build: .. / services / doctor - service
    ports:
-"3003:3003"
    environment:
PORT: 3003
      NODE_ENV: development
      DB_HOST: postgres - doctor
      DB_PORT: 5432
      DB_NAME: doctor_db
      DB_USER: admin
      DB_PASSWORD: password
      JWT_SECRET: SyOudFfvRbmoURg4Mq2N34wq
      UPLOAD_DIR: / uploads / doctor - verification
      KAFKA_BROKER: kafka: 9092
    depends_on:
-postgres - doctor
- kafka
    volumes:
```

```
-doctor_uploads:/ uploads / doctor - verification
    networks:
-healthcare - network

  # Appointment Service
  appointment - service:
    build: .. / services / appointment - service
    ports:
-"3004:3004"
    environment:
PORT: 3004
      NODE_ENV: development
      DB_HOST: postgres - appointment
      DB_PORT: 5432
      DB_NAME: appointment_db
      DB_USER: admin
      DB_PASSWORD: password
      JWT_SECRET: SyOudFfvRbmoURg4Mq2N34wq
      AUTH_SERVICE_URL: http://auth-service:3001
DOCTOR_SERVICE_URL: http://doctor-service:3003
TELEMEDICINE_SERVICE_URL: http://telemedicine-service:3005
KAFKA_BROKER: kafka: 9092
    depends_on:
-postgres - appointment
- auth - service
- doctor - service
- kafka
    networks:
-healthcare - network

  # Telemedicine Service
  telemedicine - service:
    build: .. / services / telemedicine - service
    ports:
-"3005:3005"
    environment:
PORT: 3005
      NODE_ENV: development
      DB_HOST: postgres - telemedicine
      DB_PORT: 5432
      DB_NAME: telemedicine_db
      DB_USER: admin
      DB_PASSWORD: password
      JWT_SECRET: SyOudFfvRbmoURg4Mq2N34wq
      JITSI_BASE_URL: https://meet.jit.si
depends_on:
-postgres - telemedicine
    networks:
-healthcare - network
  # Payment Service
  payment - service:
    build: .. / services / payment - service
    ports:
-"3006:3006"
    environment:
PORT: 3006
      NODE_ENV: development
      DB_HOST: postgres - payment
      DB_PORT: 5432
      DB_NAME: payment_db
      DB_USER: admin
```

```
      DB_PASSWORD: password
      KAFKA_BROKER: kafka: 9092
    depends_on:
-postgres - payment
- kafka
    networks:
-healthcare - network

  # Notification Service
  notification - service:
    build: .. / services / notification - service
    ports:
-"3007:3007"
    environment:
PORT: 3007
      NODE_ENV: development
      DB_HOST: postgres - notification
      DB_PORT: 5432
      DB_NAME: notification_db
      DB_USER: admin
      DB_PASSWORD: password
      KAFKA_BROKER: kafka: 9092
      ENABLE_SMS: "${ENABLE_SMS:-true}"
      SMSAPI_TOKEN: "367|2SaoowtSbq3Z5QnK4111TghRLMRaQZ0fgM7ZJyBF"
      SMSAPI_SENDER_ID: "SMSAPI Demo"
    depends_on:
-postgres - notification
- kafka
    networks:
-healthcare - network

  # ML Service (Python Flask - Model Training & Predictions)
  ml - service:
    build:
context: .. /
dockerfile: services / ai - symptom - service / ml - integration / Dockerfile
    ports:
-"5000:5000"
    environment:
FLASK_ENV: production
FLASK_APP: symptom_analyzer.py
networks:
      -healthcare - network
    healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 10s
      timeout: 5s
      retries: 3
      start_period: 20s

  # AI Symptom Service (Node.js API Proxy)
  ai-symptom-service:
    build: .. / services / ai - symptom - service
    ports:
-"3008:3008"
    environment:
PORT: 3008
      NODE_ENV: development
      ML_SERVICE_URL: http://ml-service:5000
DB_HOST: postgres - ai
      DB_PORT: 5432
```

```
      DB_NAME: ai_symptom_db
      DB_USER: admin
      DB_PASSWORD: password
    depends_on:
      -postgres - ai
      - ml - service
    networks:
-healthcare - network

  # PostgreSQL Databases
  postgres - auth:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: auth_db
ports:
      -"5401:5432"
    volumes:
-auth_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - patient:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: patient_db
ports:
      -"5402:5432"
    volumes:
-patient_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - doctor:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: doctor_db
ports:
      -"5403:5432"
    volumes:
-doctor_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - appointment:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: appointment_db
ports:
      -"5404:5432"
    volumes:
-appointment_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network
```

```
  postgres - telemedicine:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: telemedicine_db
ports:
      -"5405:5432"
    volumes:
-telemedicine_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - payment:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: payment_db
ports:
      -"5406:5432"
    volumes:
-payment_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - ai:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: ai_symptom_db
ports:
      -"5408:5432"
    volumes:
-ai_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  postgres - notification:
    image: postgres: 15 - alpine
    environment:
POSTGRES_USER: admin
POSTGRES_PASSWORD: password
POSTGRES_DB: notification_db
ports:
      -"5407:5432"
    volumes:
-notification_db_data:/ var / lib / postgresql / data
    networks:
-healthcare - network

  # Kafka
  kafka:
image: confluentinc / cp - kafka:7.4.0
    environment:
KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper: 2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
```

```
-"9092:9092"
    depends_on:
-zookeeper
    networks:
-healthcare - network

  # Zookeeper
  zookeeper:
image: confluentinc / cp - zookeeper:7.4.0
    environment:
ZOOKEEPER_CLIENT_PORT: 2181
    ports:
-"2181:2181"
    networks:
-healthcare - network

  # ─── Monitoring Stack ──────────────────────────────────────────────

# Prometheus - metrics collection
  prometheus:
image: prom / prometheus:v2.51.0
    ports:
      -"9090:9090"
    volumes:
- ./ monitoring / prometheus.yml:/ etc / prometheus / prometheus.yml:ro
- prometheus_data:/ prometheus
    command:
-"--config.file=/etc/prometheus/prometheus.yml"
- "--storage.tsdb.path=/prometheus"
- "--storage.tsdb.retention.time=15d"
- "--web.console.libraries=/usr/share/prometheus/console_libraries"
- "--web.console.templates=/usr/share/prometheus/consoles"
    networks:
-healthcare - network
    restart: unless - stopped

  # Grafana - metrics visualization
  grafana:
image: grafana / grafana:10.4.0
    ports:
-"3009:3000"
    environment:
GF_SECURITY_ADMIN_USER: admin
GF_SECURITY_ADMIN_PASSWORD: admin123
GF_USERS_ALLOW_SIGN_UP: "false"
    volumes:
-grafana_data:/ var / lib / grafana
- ./ monitoring / grafana / provisioning:/ etc / grafana / provisioning:ro
depends_on:
      -prometheus
    networks:
-healthcare - network
    restart: unless - stopped

  # Docker Stats Exporter - container CPU/memory/network metrics via Docker API
# Replaces cAdvisor which does not work on Docker Desktop for Windows
  docker - stats - exporter:
    build:
context: ./ monitoring / docker - stats - exporter
    ports:
-"9338:9338"
```

```
    volumes:
- / var / run / docker.sock:/ var / run / docker.sock:ro
networks:
      -healthcare - network
    restart: unless - stopped

  # Node Exporter - host OS metrics
  node - exporter:
    image: prom / node - exporter:v1.7.0
    ports:
      -"9100:9100"
    volumes:
- / proc:/ host / proc:ro
- / sys:/ host / sys:ro
- /:/ rootfs:ro
command:
      -"--path.procfs=/host/proc"
      - "--path.sysfs=/host/sys"
      - "--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($$|/)"
    networks:
-healthcare - network
    restart: unless - stopped

volumes:
auth_db_data:
patient_db_data:
patient_uploads:
doctor_db_data:
doctor_uploads:
appointment_db_data:
telemedicine_db_data:
payment_db_data:
notification_db_data:
ai_db_data:
prometheus_data:
grafana_data:

networks:
healthcare - network:
    driver: bridge
```

--5(Prometues Scraper)

```
global:
scrape_interval: 15s
evaluation_interval: 15s
external_labels:
    monitor: "healthcare-platform"

scrape_configs:
# Prometheus itself
-job_name: "prometheus"
    static_configs:
-targets: ["localhost:9090"]

# Docker Stats Exporter - container CPU/memory/network metrics
- job_name: "cadvisor"
    static_configs:
-targets: ["docker-stats-exporter:9338"]
    metrics_path: / metrics

# Node Exporter - Host OS metrics
```

```
  - job_name: "node-exporter"
    static_configs:
-targets: ["node-exporter:9100"]

# API Gateway - application HTTP metrics
- job_name: "api-gateway"
    static_configs:
-targets: ["gateway:3000"]
    metrics_path: / metrics

# Individual service health endpoints (scraped as up/down probes)
  - job_name: "auth-service"
    static_configs:
-targets: ["auth-service:3001"]
    metrics_path: / metrics

  - job_name: "patient-service"
    static_configs:
-targets: ["patient-service:3002"]
    metrics_path: / metrics

  - job_name: "doctor-service"
    static_configs:
-targets: ["doctor-service:3003"]
    metrics_path: / metrics

  - job_name: "appointment-service"
    static_configs:
-targets: ["appointment-service:3004"]
    metrics_path: / metrics

  - job_name: "telemedicine-service"
    static_configs:
-targets: ["telemedicine-service:3005"]
    metrics_path: / metrics

  - job_name: "payment-service"
    static_configs:
-targets: ["payment-service:3006"]
    metrics_path: / metrics

  - job_name: "notification-service"
    static_configs:
-targets: ["notification-service:3007"]
    metrics_path: / metrics

  - job_name: "ai-symptom-service"
    static_configs:
-targets: ["ai-symptom-service:3008"]
    metrics_path: / metrics

  - job_name: "ml-service"
    static_configs:
-targets: ["ml-service:5000"]
    metrics_path: / metrics
```

--6 Kafka Configuration

```
apiVersion: apps / v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: healthcare
```

```
spec:
  serviceName: kafka-headless
  replicas: 1
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
        - name: kafka
          image: confluentinc/cp-kafka:7.4.0
          ports:
            - containerPort: 9092
          env:
            - name: KAFKA_BROKER_ID
              value: "1"
            - name: KAFKA_ZOOKEEPER_CONNECT
              value: "zookeeper:2181"
            # Use the headless DNS name so other pods in the cluster can reach this
broker
            - name: KAFKA_ADVERTISED_LISTENERS
              value: "PLAINTEXT://kafka:9092"
            - name: KAFKA_LISTENERS
              value: "PLAINTEXT://0.0.0.0:9092"
            - name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR
              value: "1"
            - name: KAFKA_LOG_RETENTION_HOURS
              value: "168"
            - name: KAFKA_AUTO_CREATE_TOPICS_ENABLE
              value: "true"
          volumeMounts:
            - name: data
              mountPath: /var/lib/kafka/data
          readinessProbe:
            exec:
              command:
                - /bin/sh
                - -c
                - kafka-topics --bootstrap-server localhost:9092 --list
            initialDelaySeconds: 30
            periodSeconds: 15
            failureThreshold: 5
          livenessProbe:
            tcpSocket:
              port: 9092
            initialDelaySeconds: 60
            periodSeconds: 20
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
            limits:
              memory: "1Gi"
              cpu: "1000m"
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
```

```
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
---
# Headless service for the StatefulSet DNS
apiVersion: v1
kind: Service
metadata:
  name: kafka-headless
  namespace: healthcare
spec:
  clusterIP: None
  selector:
    app: kafka
  ports:
    - port: 9092
      targetPort: 9092
---
# ClusterIP used by microservices to connect to Kafka
apiVersion: v1
kind: Service
metadata:
  name: kafka
  namespace: healthcare
spec:
  selector:
    app: kafka
  ports:
    - port: 9092
      targetPort: 9092
```

Appendix B- Kariyawsam L.L.N

Appendix C – Bandara A.M.H.V.R

Appendix D- Budara V.P.R